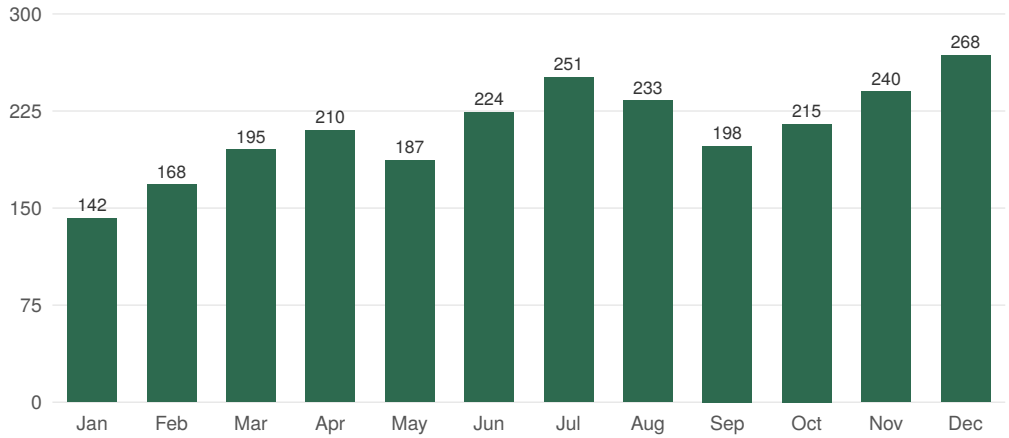
Monthly Sales 2025

This report visualises sales values entirely server-side: a Lua block below computes scaling, axes and bar geometries from a plain Lua table, writes the result to `chart.svg`, and glu inlines it into the PDF as a regular image. No JavaScript engine, no headless browser, no external chart library — just data in, SVG out, PDF rendered.

The same pattern scales to JSON feeds, database queries or aux values from earlier passes. Because the SVG is a pure function of the input data, identical inputs produce byte-identical PDFs.



How it works

The Lua block above runs during Markdown processing. Its return value replaces the block in the source, so the generated `<img>` tag is what goldmark sees. htmlbag then loads `chart.svg` from disk and embeds it as a vector graphic in the final PDF.

Three glu/Lua features carry the workflow:

1. **Lua blocks in Markdown** (```` ```{lua} … ``` ````) — anything that returns a string substitutes into the document.
2. **Full Lua standard library** — `io.open`, `string.format`, `table.concat` are all available in the default (non-`---safe`) mode.
3. **htmlbag SVG support** — any `<img src="*.svg">` is rendered as vector content, not rasterised.

What about JavaScript-based charts?

For traditional dashboards driven by Chart.js or D3, the usual approach is to render the chart in a headless browser and snapshot it. Pipelines like the one above sidestep that entirely: no Chromium process, no race on "when did the chart finish drawing", no 150 MB of Chromium in the deployment image. The chart is just data, deterministically serialised to SVG, and the rendering happens in glu itself in a few milliseconds.